

Team 11

ROAMIE

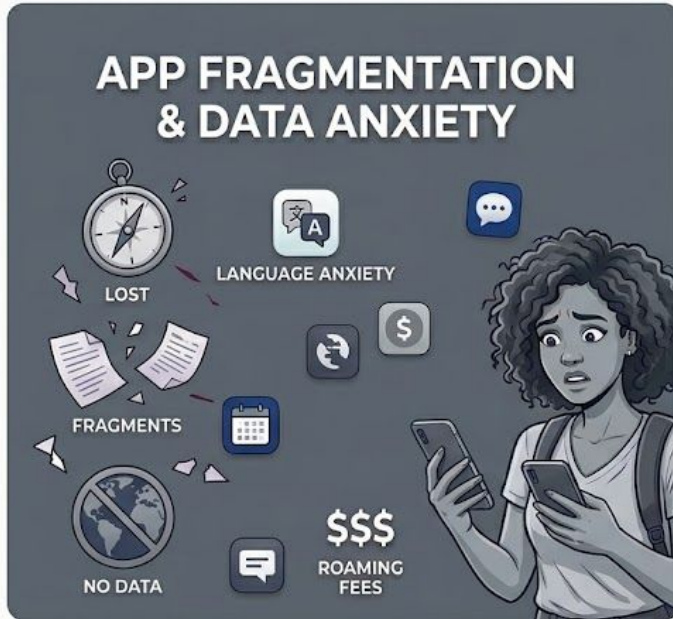


Outline

- Intro
- Background and problem we are trying to solve
- Go over the specifications of our app
- Demo – show the clips and talk over it – show the entire user flow
 - Auth -> Explore -> Trip Creation -> Map -> AI Assist -> Itinerary page
 - Profile Page -> edit profile -> social connect -> messaging
 - AI Assist plan mode
 - Show other features on explore page
 - Show navigation feature + offline features
- Discuss the tech stack, design consideration
- How did we utilize AI/LLMs for development – Cursor, cursor rules, AI code review, etc
- Frontend – UI design and Collecting analytics for frontend
- DevOps – Deploying the backend, logging data, CDN, object storage, github action
- Scalability strategy
- Challenges faced and what did we learn
- Conclusion – were we able to achieve our objectives, whats next
- Questions
- Thank you

Background and Problem

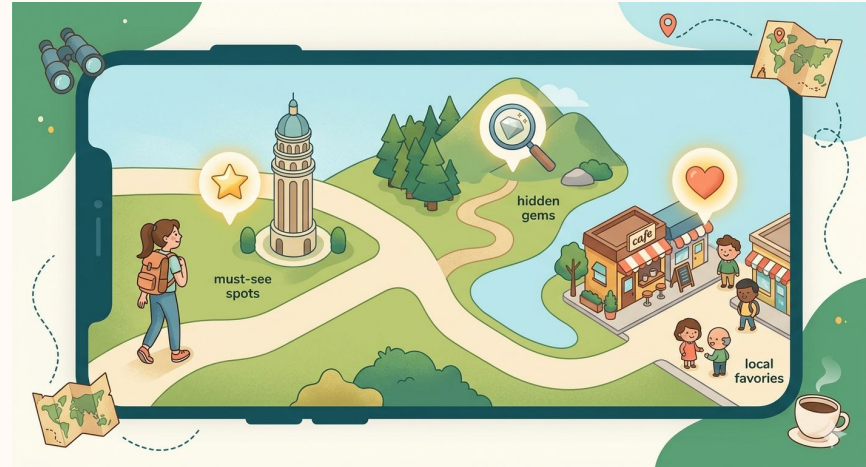
THE TRAVEL DISCONNECT: CURRENT CHAOS vs. UNIFIED EXPERIENCE



Specifications

Core Features

- Itinerary planning
- Interactive offline maps with place pins
- Curated local recommendations
- Language phrasebook with pronunciation
- On-device AI travel assistant
- Offline first
- Social connect



Live Demo

Tech Stack



Frontend

React Native with Expo SDK 55, NativeWind for styling, Zustand for state management, and expo-router for navigation



Data Layer

PostgreSQL for relational data, SQLite for offline storage, Upstash Redis for caching and rate limiting, and Cloudflare/Backblaze B2 for image hosting



Backend

ElysiaJS with Bun runtime, Eden Treaty, Prisma ORM for PostgreSQL, TypeBox for validation, JWT for authentication, and OpenTelemetry



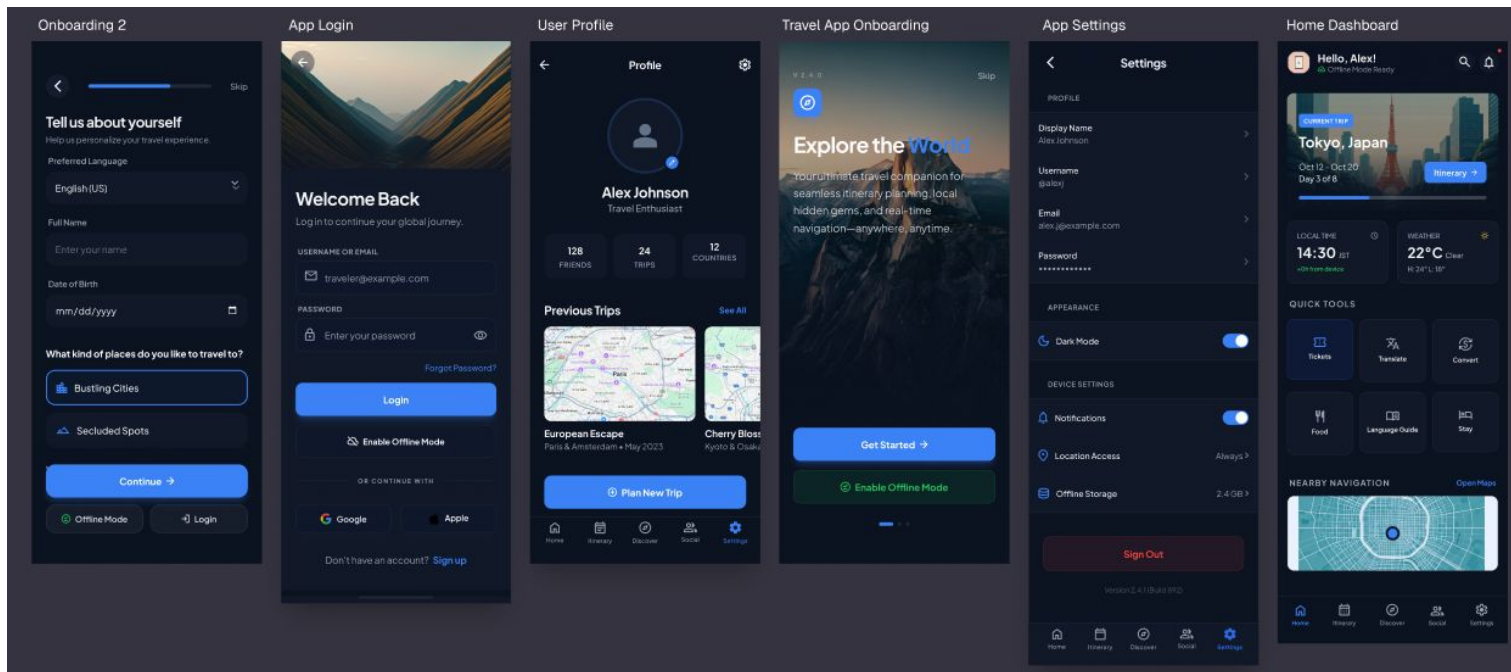
AI Engine

Gemma-2-2b-it-GGUF model running on-device via llama.rn, no server inference required

AI Assisted Development

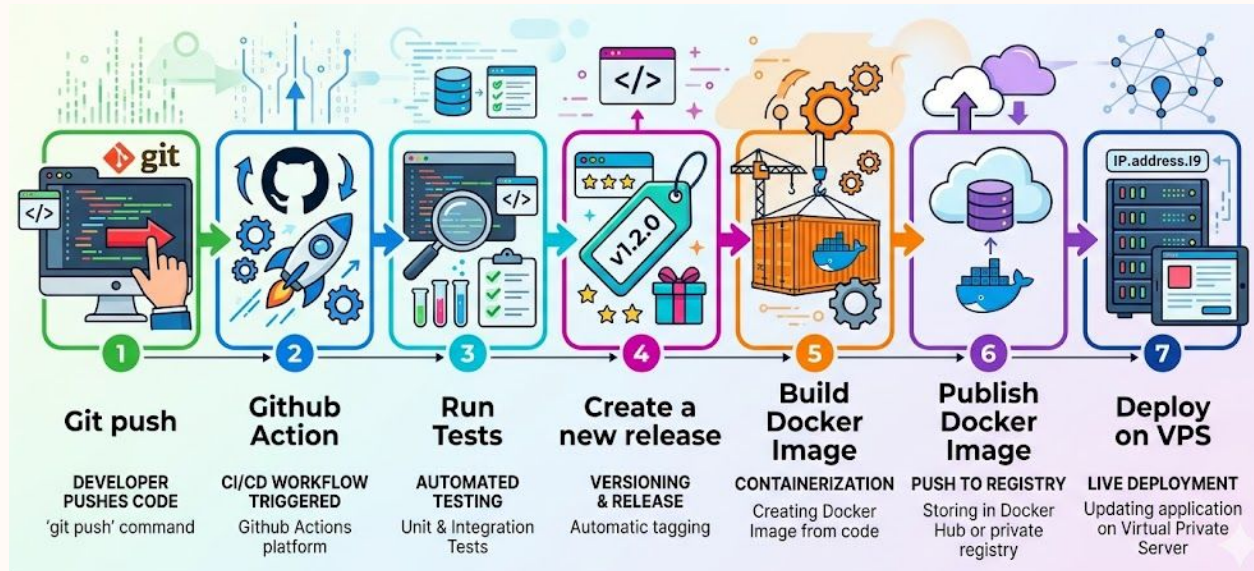


Early UI Designs



DevOps

Backend Build/Deploy Pipeline



✓ API Testing

1m 5s

✓ Semantic Release (API)

26s

✓ Build and Push Docker I...

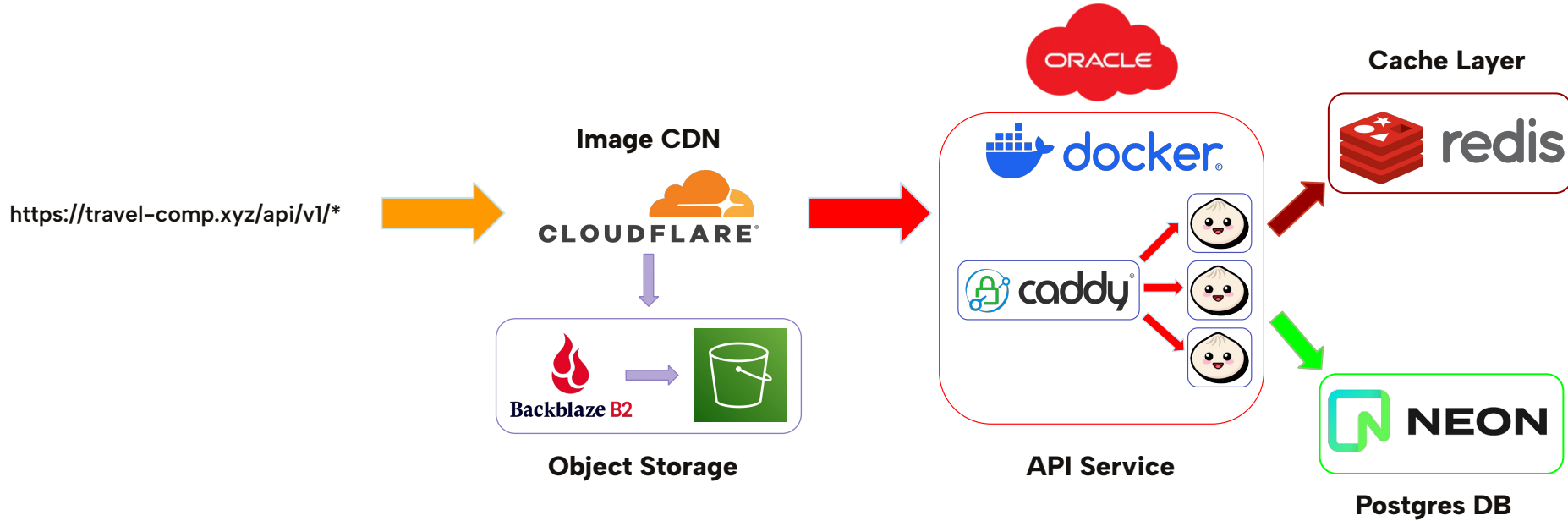
9m 17s

✓ Deploy on VPS

23s

DevOps

Backend Architecture



Scalability Strategy

Backend Deployment

AWS EKS with Kubernetes auto-scaling across multiple availability zones

Database Scaling

Neon PostgreSQL with read replicas and connection pooling via PgBouncer

Cache Strategy

Upstash Redis for JWT blocklist, rate limiting, and query caching with 12-hour TTL

WebSocket Chat

Upstash Redis Pub/Sub bridges connections across pods for real-time messaging

Estimates

100K

Concurrent Users

\$2.5K

Monthly Cost

3

Regions

30+

Pod Replicas

Challenges and Lessons Learned

Offline-First Complexity

Managing conflict resolution between client and server required careful design of outbox queue and sync engine

On-Device AI

Running LLM locally meant balancing model size (1.7 GB) with performance while maintaining offline capability

Multi-Platform OAuth

Implementing Google OAuth for Android and Apple OAuth for iOS required platform-specific code paths

Map Integration

Offline tile packs and pin rendering demanded efficient memory management to prevent crashes

Conclusion and Next Steps

Achievements:

- Built a production ready, offline first, travel companion mobile app in 10 weeks
- Implemented sophisticated features and design a scalable backend
- Integrated an on-device LLM for travel assistance
- Everyone was able to experience a lead role and work in a fast paced environment with AI.

Future Plans:

- Improve the current features and add some new features like live translation
- Optimize the app to have a lower memory and battery footprint
- Create city packs for many more destinations and integrate Google Maps Places API
- Allow multiple users to interact with a single itinerary



Questions?

Thank you